Studio Nx

Backend(FastAPI/Python) 18 — release_withdrawn()

2026-10-07

Backend(FastAPI/Python) 18 — `release_withdrawn()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 탈퇴 후 재가입 · PRD 01 FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	17 `normalize_email()` · 19 `check_email()`

1. 하는 일

탈퇴한 회원의 이메일 · 닉네임을 **다른 값으로 바꿔서**, 같은 이메일 · 닉네임으로 새로 가입할 수 있게 한다.

칸	바꾸기 전 (탈퇴한 85번 회원)	바꾼 뒤
`email`	`kim@naver.com`	`deleted+85@withdrawn.invalid`
`nickname`	`beatmaker`	`deleted_user_0000000085`
`status` · `deleted_at` · 프로젝트		**그대로**

왜 필요한가

순서	일어나는 일
①	85번 회원이 탈퇴 → 행은 지우지 않고 `status=DELETED` (P1-09, 소프트 삭제)
②	행이 `kim@naver.com` · `beatmaker` 를 그대로 쥐고 있음
③	4일 뒤 같은 사람이 `kim@naver.com` 으로 다시 가입 → `email` 은 `UNIQUE` 라 DB 가 거절
④	그래서 ③ 직전에 이 함수가 85번 행의 값을 비켜 둠 → 새 행이 들어갈 수 있음

2. 언제 쓰나

"그 값으로 다시 가입하려는 사람이 나타났을 때" 부른다. 탈퇴 3일이 지났는지는 **부르는 쪽이 확인**한다.

쓰는 곳	화면	상황
19 `check_email()` · 20 `check_nickname()`	join.html 중복 확인	그 값을 쥔 행이 탈퇴 후 3일이 지났으면 비켜 두고 "쓸 수 있음"
21 `signup()`	join.html Sign Up 버튼	가입 직전 같은 확인

탈퇴 후	결과 (FR-108, 2026-10-03 결정)
3일 안	비켜 두지 않음 → "You can sign up again after ..."
3일 지남	**이 함수** → 새 가입 가능

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. 이 함수는 **`Session`** · **`User`** 를 타입 표시에 쓴다.

함수

```
def release_withdrawn(db: Session, user: User) -> None:
    user.email = f"deleted+{user.user_id}@withdrawn.invalid"
    user.nickname = f"deleted_user_{user.user_id:010d}"
    db.flush()
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻	85번 회원일 때
`f"deleted+{user.user_id}@…"`	f-문자열. `{ }` 안의 값을 글자에 끼움	`deleted+85@withdrawn.invalid`
`{user.user_id:010d}`	정수(`d`)를 **10자리**로, 빈자리는 `0` 으로 채움	`0000000085`
`db.flush()`	바꾼 값을 지금 DB 에 보냄 (`UPDATE`). 아직 확정(`commit`)은 아님	
`-> None`	돌려주는 값이 없음	

4. 흐름

1. 이메일을 `deleted+<번호>@withdrawn.invalid` 로 바꾼다.
2. 닉네임을 `deleted_user_<번호 10자리>` 로 바꾼다.
3. `flush` 로 DB 에 먼저 반영한다 → 같은 저장 안에서 이어 넣는 새 행이 `UNIQUE` 에 걸리지 않는다.

5. 왜 이렇게

선택	이유
행을 지우지 않고 값만 바꿈	탈퇴 회원이 남긴 프로젝트(`owner_id`) · 문의 기록이 그 행을 가리킨다. 지우면 FK 때문에 지울 수도 없다
회원 번호를 넣음	탈퇴 회원마다 값이 달라서 비켜 둔 값끼리도 `UNIQUE` 에 걸리지 않는다
이메일 도메인 `.invalid`	인터넷 규격(RFC 2606)상 **실제로 존재할 수 없는** 도메인이다. 이 주소로 메일이 갈 일이 없다
닉네임을 템플릿(`deleted_85`)과 다르게 **23자**로	`deleted_85` 는 회원이 직접 가입할 수 있는 닉네임(2 ~ 20자)이다. 누가 먼저 그 이름을 쓰고 있으면 비켜 둘 때 `UNIQUE` 충돌 → 500. 가입 닉네임은 최대 20자라 21자 이상인 이름은 **절대 겹치지 않는다** (칸은 50자까지)
`flush` (`commit` 아님)	가입과 한 저장으로 묶는다. 가입이 실패하면 비켜 둔 것도 함께 취소된다

선택	이유
3일 확인은 부르는 쪽에서	이 함수는 "비켜 두기" 한 가지만 한다. 언제 비켜 둘지는 19 ~ 21 이 정한다

탈퇴 회원 추적 방지 (2026-10-07 결정)

비켜 둔 값에는 **회원 번호만** 들어 있다. 원래 이메일 · 닉네임은 덮어써져 남지 않으므로, 이 값으로 그 사람이 누구였는지 알아낼 수는 없다. 회원 번호 자체도 바깥에 드러나지 않게 아래를 지킨다.

대책	언제 · 어디서	막는 것
① 다른 사람에게는 **'Deleted user'** 로만 보냄. 그 사람의 `user_id` 도 응답에 넣지 않음	Phase 6 멤버 목록 · 채팅 · 댓글 (`user_service.withdraw` TODO 에 기록)	`deleted_user_0000000085` 가 화면에 보여 회원 번호가 드러나는 것
② 3일 뒤 남은 개인 정보도 지움: `password_hash = "!"` · `last_login_at` · `profile_image_path` 비움	보관 기간 정리 `retention.py` (P1-09, TODO 에 기록)	탈퇴한 행에 개인 정보가 계속 남는 것
③ 회원 번호로 다른 회원을 찾는 공개 API 를 만들지 않음	지금 API 확인함 — 회원 번호는 가입 응답 · 내 정보(본인)에만 나감	번호를 알아도 그 번호로 아무것도 조회할 수 없게

DB 를 직접 보는 관리자(SQLAdmin, P1-21)는 회원 번호와 탈퇴 행을 볼 수 있다. 관리 목적상 필요하고, 관리자 화면은 ADMIN 계정만 들어간다.

6. 시험 (`tests/test_auth_service.py` 에 4개 추가 · 모두 11개, 실제 MySQL · rollback)

시험	확인한 것
비켜 둔 뒤 같은 값으로 가입	이메일 · 닉네임이 바뀌고, 같은 `gone@example.com` · `gone_nick` 으로 새 회원이 들어감
비켜 두지 않으면	같은 이메일로 넣을 때 DB 가 거절 (`IntegrityError`) — 이 함수가 필요한 이유
행 · 프로젝트는 그대로	`status=DELETED` · `deleted_at` 그대로, 남긴 프로젝트의 `owner_id` 그대로
닉네임 충돌 없음	비켜 둔 닉네임이 20자보다 길고 50자 이하. 회원이 `deleted_85` 같은 닉네임을 써도 충돌 없음

전체 시험 119개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.